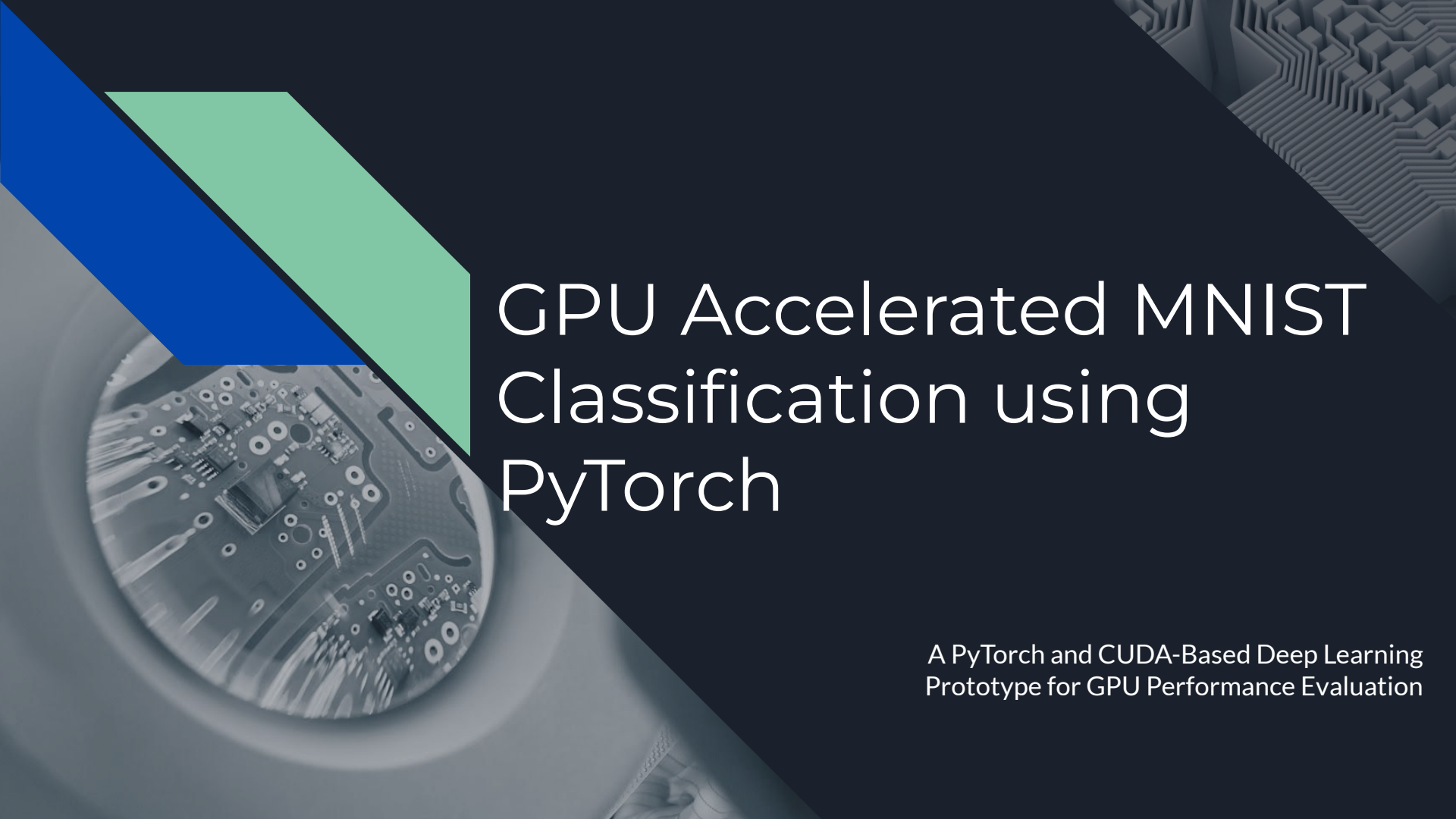


GPU Accelerated MNIST Classification using PyTorch

A PyTorch and CUDA-Based Deep Learning Prototype for GPU Performance Evaluation

Project Overview

GPU Accelerated MNIST Classification using PyTorch

Project Overview

This project demonstrates GPU acceleration for deep learning training using PyTorch and CUDA.

A neural network was developed to classify handwritten digits from the MNIST dataset and to compare CPU and GPU training performance.

Technologies

-Python 3
-PyTorch
-Torchvision
-Numpy
-Pandas
-Matplotlib
-Seaborn
-Scikit - learn
-NVIDIA CUDA Tesla T4 GPU
-MNIST Dataset
-Google Colab

Project Objectives

The main objectives of this project are:

-Implement a neural network for handwritten digit classification.

-Use GPU acceleration through CUDA and PyTorch.

-Compare CPU and GPU training performance.

-Evaluate accuracy and computational efficiency.

Key Questions:
-How does GPU acceleration impact neural network training performance?

Technologies & Environment

- **Programming Language**: Python

- **Deep Learning Framework**: PyTorch

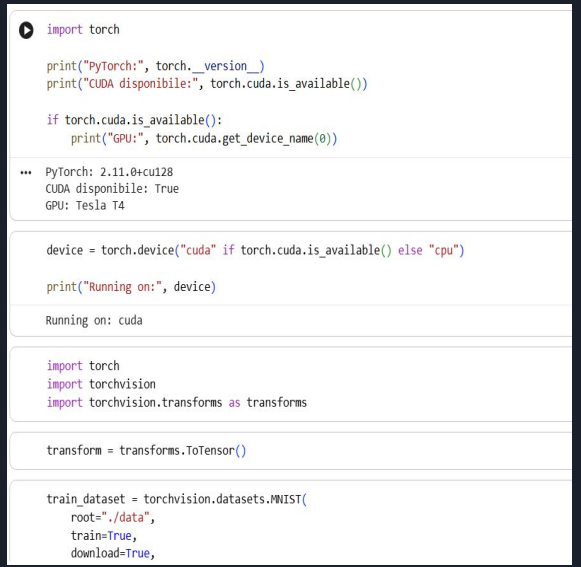
-**GPU Acceleration**: CUDA through PyTorch

- **Data Processing & Visualization**: Pandas, Seaborn, Scikit-learn, Matplotlib

- Environment: Google Colab

-**Hardware**: NVIDIA Tesla T4 GPU

-**Dataset**: MNIST handwritten digits dataset



```
import torch

print("PyTorch:", torch.__version__)
print("CUDA disponibile:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
PyTorch: 2.11.0+cu128
CUDA disponibile: True
GPU: Tesla T4
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Running on:", device)
```

```
Running on: cuda
```

```
import torch
import torchvision
import torchvision.transforms as transforms
```

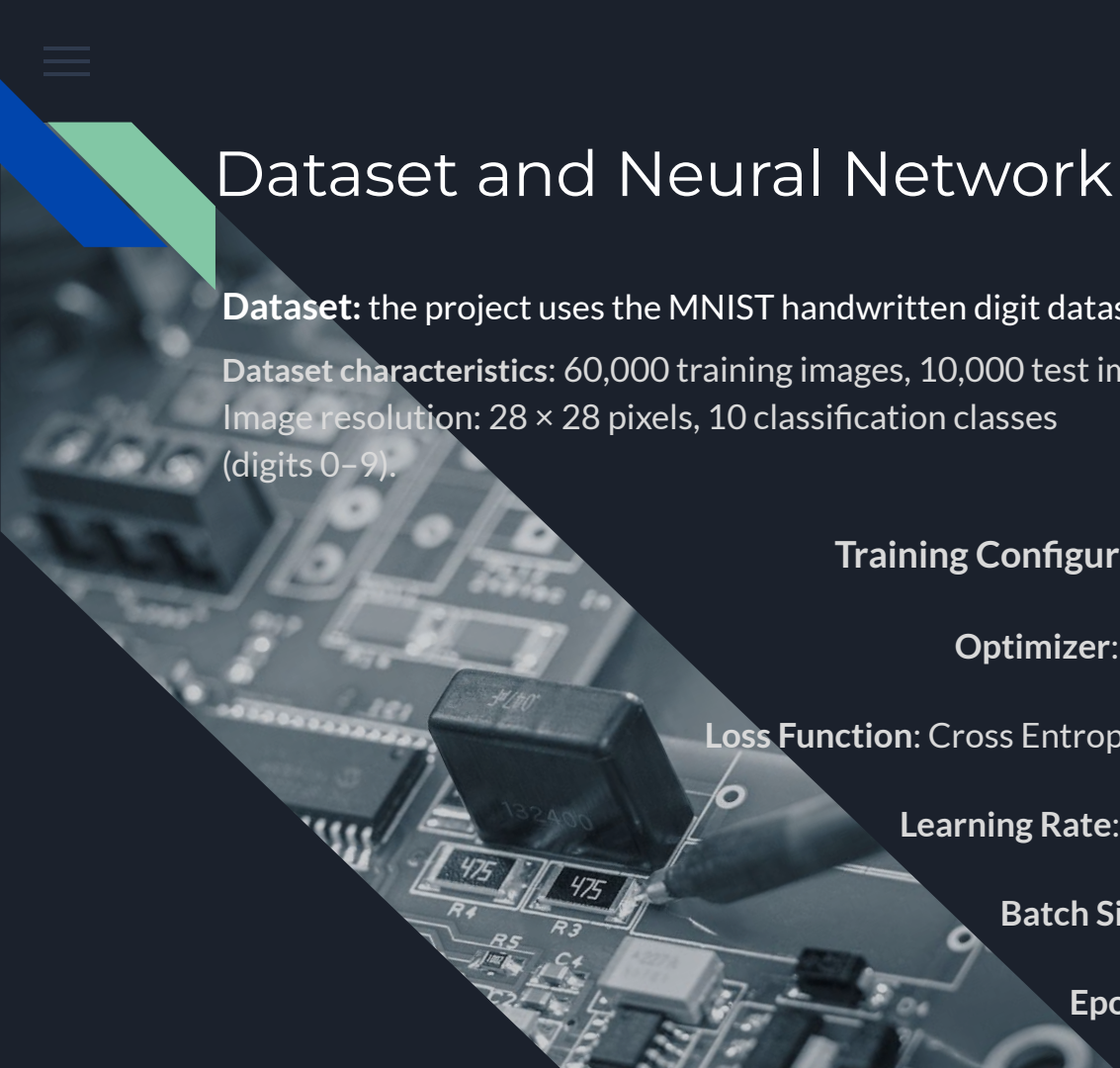
```
transform = transforms.ToTensor()
```

```
train_dataset = torchvision.datasets.MNIST(
    root="./data",
    train=True,
    download=True,
```

Dataset and Neural Network Architecture

Dataset: the project uses the MNIST handwritten digit dataset

Dataset characteristics: 60,000 training images, 10,000 test images, Image resolution: 28 × 28 pixels, 10 classification classes (digits 0–9).



Training Configuration:

Optimizer: Adam

Loss Function: Cross Entropy Loss

Learning Rate: 0.001

Batch Size: 64

Epochs: 5

GPU Implementation and Training

GPU Implementation:

PyTorch was configured to use CUDA acceleration.

```
import torch

print("PyTorch:", torch.__version__)
print("CUDA disponibile:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
PyTorch: 2.11.0+cu128
CUDA disponibile: True
GPU: Tesla T4
```

```
training_time = train(
    model,
    train_loader,
    criterion,
    optimizer,
    device,
    epochs=5
)

print(f"Training time: {training_time:.2f} seconds")
```

```
Epoch 1/5 - Loss: 0.3455
Epoch 2/5 - Loss: 0.1423
Epoch 3/5 - Loss: 0.0980
Epoch 4/5 - Loss: 0.0741
Epoch 5/5 - Loss: 0.0568
Training time: 40.72 seconds
```

```
accuracy = evaluate(
    model,
    test_loader,
    device
)

print(f"\nAccuracy: {accuracy:.2f}%")
```

```
Accuracy: 97.62%
```

```
import pandas as pd

results = pd.DataFrame({
    "GPU": ["NVIDIA Tesla T4"],
```

[1] ✓ 0s

```
!nvidia-smi
```

```
Sat Jul 25 18:42:35 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   48C    P8              9W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

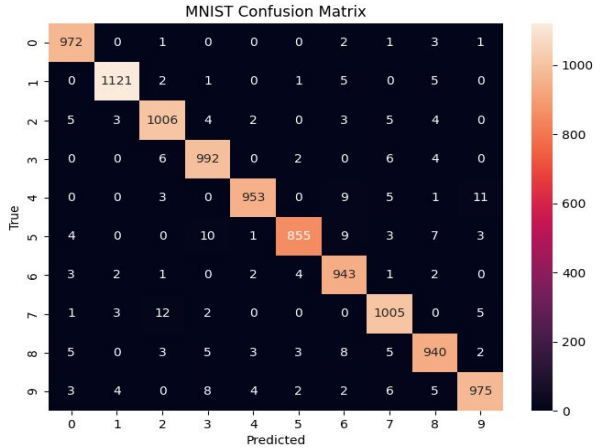
[2] ✓ 7s

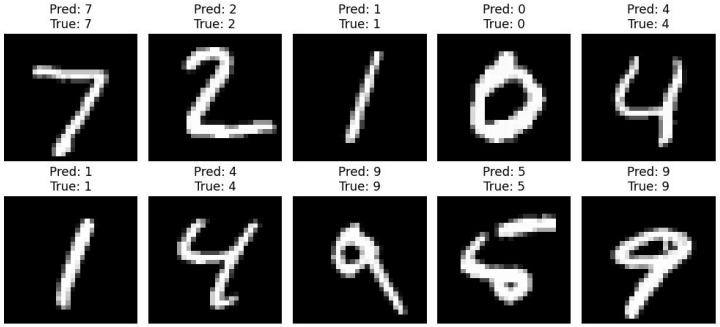
```
import torch

print("PyTorch:", torch.__version__)
print("CUDA disponibile:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

During training:
The neural network model was transferred to GPU memory,
Input tensors were processed on the NVIDIA GPU.
CUDA enabled parallel computation during model training.

Model Evaluation

The trained neural network was evaluated using:
-Classification accuracy
-Prediction visualization
-Confusion matrix analysis
-Generated outputs:
-Predicted digit examples
-Confusion matrix
-CSV performance report
-Saved trained model





CPU vs GPU Performance Comparison

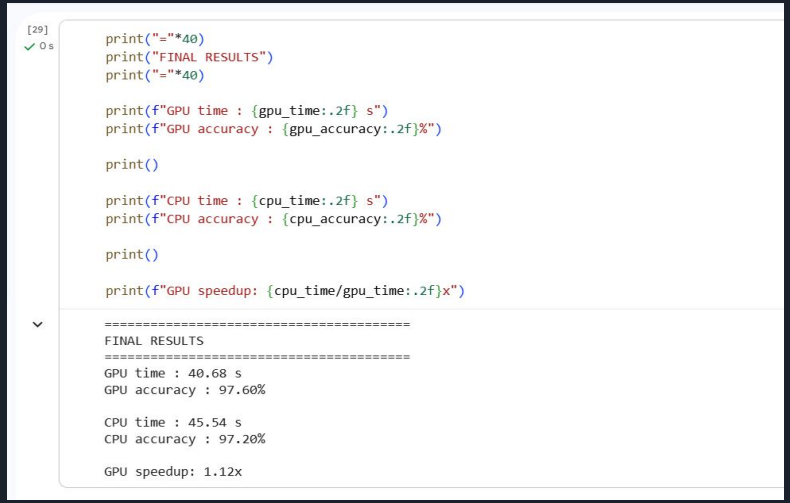
GPU Speedup

Results

Device	Training Time	Accuracy
GPU	40.68 seconds	97.60%
CPU	45.54 seconds	97.0%

GPU Speedup

1.12× faster training



```
[29]
✓ 0 s
print("="*40)
print("FINAL RESULTS")
print("="*40)

print(f"GPU time : {gpu_time:.2f} s")
print(f"GPU accuracy : {gpu_accuracy:.2f}%")

print()

print(f"CPU time : {cpu_time:.2f} s")
print(f"CPU accuracy : {cpu_accuracy:.2f}%")

print()

print(f"GPU speedup: {cpu_time/gpu_time:.2f}x")

========================================
FINAL RESULTS
========================================
GPU time : 40.68 s
GPU accuracy : 97.60%

CPU time : 45.54 s
CPU accuracy : 97.20%

GPU speedup: 1.12x
```

The moderate speedup is expected because the MNIST dataset and the fully connected neural network architecture are relatively small. Larger models and datasets can better exploit NVIDIA GPU parallel processing.

NVIDIA GPU Speedup

CPU vs GPU Training Time

GPU Speedup: 1.12×

GPU Accelerated MNIST Classification using PyTorch

Andra Corina Leustean